# First Assignment

The main goal of the first assignment is to explore and evaluate clustering algorithms alongside Large Language Models (LLMs) to extract and describe the architecture of an existing software system.

**The aim is to answer the two research questions (RQs):**

1. How do different clustering algorithms vary in their ability to accurately determine the architectural components of a system?
2. How do different prompting techniques impact LLM's ability to accurately describe architectural components based on source code?

Each group is assigned a specific part within a Java project. Table I below outlines these group assignments and provides the necessary project details. For the exact file paths of your assigned components, please refer to `[1]`.

**Table I: Group projects and Repos**

| Group | Project | Part under focus | code |
|---|---|---|---|
| Groups 1 & 6 & 11 | Tika | Detect & parser | github.com/apache/tika |
| Groups 2 & 7 & 12 | JClouds | Azureblob | github.com/apache/jclouds |
| Groups 3 & 8 & 13 | Hadoop MapReduce | Client core | github.com/apache/hadoop |
| Groups 4 & 9 & 14 | Lucene | Codecs | github.com/apache/lucene |
| Groups 5 & 10 & 15 | Hadoop Yarn | Resource manager, scheduler, capacity | github.com/apache/hadoop |

To answer the research questions, we will follow a phased workflow over the coming weeks:

## Week 1: Class Dependency Extraction

1. Compile your assigned Java project to generate the compiled *.jar file.

2. Extract system-wide dependencies: Use the ARCADE JavaParser tool to extract all project relationships and generate a master .rsf file. For the documentation on the ARCADE tool suite, refer to [2].

3. Filter the dependencies to isolate classes strictly relevant to your group's assigned component (e.g., Server API), resulting in a focused *.rsf file.

   A standard *.rsf file lists dependencies line-by-line in the format:

   ```
   depends Source_Package Target_Package
   ```

   Filtering here means extracting only the lines where either the source or the target packages are in the part under focus of your assigned project.

4. Cluster and Tune: Use the ARCADE Clusterer and ARCADE ACDC tools to run the WCA, Limbo, and ACDC (structural) clustering algorithms. Adjust the tool's parameters, if applicable, as needed to optimize your results and generate the final clustered *.rsf output [2].

## ✨Week 2: Clusters evaluation & Initial exploration of LLMs

1. **Conduct a comparative evaluation across applied clustering approaches from Week 1.**

Use the ARCADE a2a (Architecture-to-Architecture) and Cvg (Coverage) metric tools to measure the structural similarity and distance between the two architectures (.rsf outputs (generated from your clustering phase) .

Provide the relative file paths for the respective .rsf outputs (generated from your clustering phase) [2].

Calculate the a2a and cvg among all pairs of clustering algorithms. Determine which clustering algorithm produces most similar clusters.

## 2. **Prepare LLMs for prompting.**

In this step, you will explore and prepare LLMs for prompting.

Each group is assigned specific lightweight & Heavyweight LLMs. Table II below outlines these group LLMs and their full path where they are hosted in the Hugging Face (HF) platform.

**Table II: Group light- and heavy-weight LLMs**

| Group | Lightweight Models<br><br>(Colab / T4 GPU) | Heavyweight Models<br><br>(HPC / 2x A100) |
|---|---|---|
| **Groups 1 & 6 & 11** | Qwen/Qwen2.5-7B-Instruct | Qwen/Qwen2.5-72B-Instruct |
| **Groups 2 & 7 & 12** | deepseek-ai/DeepSeek-R1-Distill-Qwen-7B | deepseek-ai/DeepSeek-V4-Pro |
| **Groups 3 & 8 & 13** | ibm-granite/granite-3.3-8b-instruct | ibm-granite/granite-34b-code-instruct-8k |
| **Groups 4 & 9 & 14** | mistralai/Mistral-7B-Instruct-v0.3 | mistralai/Mistral-Large-3-675B-Instruct-2512 |
| **Groups 5 & 10 & 15** | ByteDance-Seed/Seed-Coder-8B-Instruct | zai-org/GLM-5 |

**First: Prototyping in Google Colab (Browser), suitable for lightweight models**

Use the provided Colab notebook [3] to verify your prompt logic before scaling to the cluster.

- Open the provided notebook link and select **File > Save a copy in Drive**.
- Do **not** paste your HF token into a code cell. Instead, click the **Key icon (Secrets)** in the left sidebar, add a new secret named `HF_TOKEN`, and paste your token as the value.
- Modify the `model_name` variable to match your group's assigned **Lightweight** model
- Follow the instructions in the script.

**Second: Production Scaling on the (HPC) (Slurm Jobs)**
***[Needs access to university HPC]*** ⏳

Move to the university High Performance Computer (HPC) [4] cluster to process the full part of your assigned project.

- **Script Modification:** Update the `model_name` in your `sample.py` to your assigned **Heavyweight** model.
- **Token Handling:** The provided `sample.sh` is designed to read your HF token from a local environment variable.
- **Execution:** Ensure you follow the instructions in the comments of the provided scripts [5] to provide your token securely during the `sbatch` submission.

---

**[1] The provided Projects_Parts_under_focus.xlsx**

**[2] The provided Arcade documentation.pdf**

**[3]** [READ-ONLY]DS4SE26Week2.ipynb

**[4]** UPB PC2 WiKi

**[5] The provided Slurm sample** ⏳

---

## Tentative Project Roadmap (Subject to Adjustment)

*Note: The following workflow outlines our goals for the upcoming weeks and will be written in detail and may be refined as the project progresses.*

**Week 3: Semantic Clustering & Evaluation. Apply semantic clustering algorithms and once again conduct a comparative evaluation across the different clustering approaches.**

**<u>Week 4:</u> LLM-Based Architectural Recovery. Apply different LLMs prompting techniques to generate architectural titles and a high-level descriptive summary for each identified cluster and conduct a qualitative assessment of these AI-generated descriptions.**

**<u>Week 5</u>: Finalize your group's research report and presentation.**